

1. DATE DE IDENTIFICARE

Acronim proiect:	MENTORA
Titlu complet:	Platforma Educationala Interactiva pentru Programare
Sectiune:	C - Tehnologia Informatiei
Categorie:	Seniori (clasele IX-XII)
Elev 1:	Haivas Eduard Andrei - backend server Java, protocol binar, baza de date
Elev 2:	Perjoc Eduard Mihai - client Unity (C#), aplicatie Android, editor web
Mentor:	Ramona Radulescu
Institutie mentor:	Asociatia IPV - Informatica pentru Viitor
Telefon mentor:	+40 768 901 199
Colaboratori:	—
An scolar:	2025-2026

2. REZUMATUL PROIECTULUI

Mentora este o platforma educationala completa destinata elevilor care doresc sa invete programare in Python si C++. Sistemul combina patru componente ce conlucreaza in timp real: un joc 3D in Unity HDRP, o aplicatie Android pentru parinti, un editor web pentru profesori si un server Java/Spring Boot care orchestreaza toata comunicarea.

Elevul rezolva provocari de programare integrate in decorul jocului. Codul este executat pe server in sandbox securizat (izolare retea prin unshare, limite de memorie si procese via ulimit), evaluat de LLaMA-3.3-70b (API Groq) si integrat intr-un profil de invatare JSONB in PostgreSQL. Parintele urmareste statistici, seteaza obiective cu recompense si monitorizeaza sesiunile din aplicatia Android. Comunicarea foloseste protocol binar WebSocket AES-256-CBC cu seed unic per cadru, iar autentificarea suporta si flux QR code scanat din joc.

3. DESCRIEREA DETALIATA A PROIECTULUI

3.1 Scop

Scopul proiectului este de a reduce bariera de acces in programare prin transformarea procesului de invatare intr-o experienta de joc captivanta, sustinuta de inteligenta artificiala adaptiva si supraveghere parentala in timp real. Platforma adreseaza absenta unui instrument integrat care sa uneasca jocul 3D, evaluarea automata LLM, urmarirea progresului si crearea de continut intr-un ecosistem coerent.

3.2 Obiective

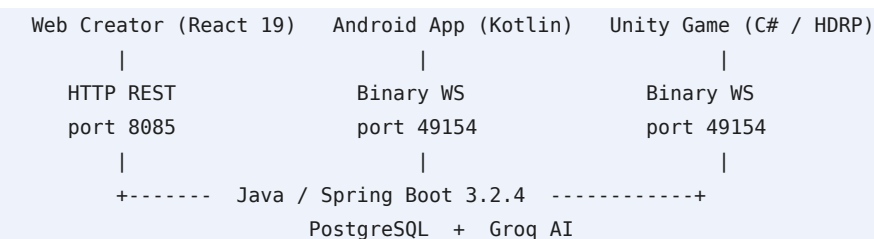
- Crearea unui joc 3D (Unity HDRP) cu provocari interactive Python si C++ integrate organic in naratiunea jocului.
- Server central Java/Spring Boot: conexiuni WebSocket binare criptate + API REST, gestionate simultan.
- Executarea in siguranta a codului arbitrar al elevilor in sandbox izolat (unshare, ulimit).
- Evaluare automata prin LLaMA-3.3-70b si profil de invatare adaptiv per elev (3 sub-profiluri JSONB).
- Dashboard parental Android/Kotlin: statistici, obiective cu recompense, monitorizare sesiuni.
- Editor web React 19 pentru profesori: publicare cursuri cu intrebari accesibile din joc.

3.3 Problema identificata si stadiul actual in domeniu

Platformele existente (Scratch, Code.org, LeetCode, CodeCombat) sufera de limitari majore: absenta jocului 3D imersiv, lipsa evaluarii adaptive prin LLM, absenta controlului parental integrat sau imposibilitatea crearii de continut de catre profesori. CodeCombat nu permite executia reala de cod nativ si nu ofera feedback pedagogic bazat pe IA.

Mentora integreaza toate aceste componente: executie reala de cod, evaluare LLM contextualizata, profil de invatare evolutiv si control parental – conectate printr-un protocol securizat.

3.4 Arhitectura sistemului



Componente si responsabilitati

java-server/	Backend central: Spring Boot 3.2.4, Java 21. HTTP :8085, WebSocket :49154.
unity/	Client joc: Unity 2022.3 HDRP, C#. Packet dispatch, UI provocari cod, QR login.
kotlin-app/	Dashboard parental: Android Kotlin + Jetpack Compose. Target SDK 36, Min SDK 24.
web-creator/	Editor cursuri: React 19 + Vite 7. SPA cu auth, CRUD cursuri, editor intrebari.
PostgreSQL	BD relationala: Parent, Child, Task, Goal, Course, GameSession, game_stats JSONB.
Groq API	LLM extern (LLaMA-3.3-70b): evaluare cod, feedback pedagogic, rezumate profil.

Fisiere-cheie server

ClientHandler.java	Dispatcher central: switch expression pe ~44 tipuri de pachete.
Server.java	Singleton: ciclu de viata WebSocket, servicii Spring, stare QR.
Packet.java	Clasa de baza: criptare/decriptare AES-256-CBC.
LearningProfileService.java	Profiluri JSONB: actualizare la fiecare eveniment, rezumare LLM.
GroqAI.java	Wrapper Groq: LRU cache 200 intrari TTL 5 min, cooldown 60s.
PythonExecutor.java	Executie Python in sandbox: unshare --net, ulimit -v/-t/-f/-u.



Fig. 1 - Harta completa a jocului 3D



Fig. 2 - Vedere aeriana cu markeri task-uri

3.5 Protocolul binar WebSocket si serverul Java

Format cadru AES-256-CBC

```
[ 4 bytes : lungime seed criptat ]
[ N bytes : seed criptat cu cheia de baza ] <- derivat din System.nanoTime()
[ M bytes : payload criptat cu seed-ul dec. ]
```

Fiecare cadru are IV (seed) unic, prevenind atacurile de replay. Seed-ul este criptat cu o cheie de baza partajata; payload-ul este criptat cu seed-ul decriptat.

Pachete (44 tipuri) - clasificare functionala

Auth & Sesiune:	HandShake(1), Auth(2), Register(3), ChildLogin(11), VerifySession(13), GameSessionStart(15), GameSessionEnd(32)
QR Login:	QRGenerate(19), QRScan(25), QRApprove(41), QRPoll(43), QRApproveResponse(44)
Copii & Obiective:	AddChild, FetchChildren, AddGoal, FetchGoals, CompleteGoal
Taskuri:	AddTask, FetchTasks, CompleteTask, FetchTasksByParent
Statistici:	FetchChildStats (actualizeaza streak), FetchChildStatsByParent (nu modifica streak)
Cod & IA:	ExecutePythonCode, ExecuteCPPCode, AskAi, AiResponse
Cursuri:	FetchPublishedCourses, FetchCourseDetail, SubmitCourseCompletion

Whitelist pre-autentificare

Pachetele 1, 2, 3, 11, 13, 15, 19, 25, 32, 41, 43, 44 sunt permise inainte de autentificare. Orice alt pachet primeste `ActionResponse(false, "Unauthorized")`. Verificarea se face in `ClientHandler.java` inainte de intrarea in switch-ul de dispatch.

Push in timp real

Cand un parinte adauga un obiectiv din aplicatia Android, serverul identifica imediat copilul online in `ConcurrentHashMap`-ul conexiunilor active si ii trimite proactiv `FetchGoalsResponse` actualizat, fara ca jocul sa fi solicitat explicit datele.

3.6 Sistemul de taskuri si obiective

Task Goal	Copilul completeaza un task specific (matching dupa titlu vs. <code>DefaultTaskType</code> enum).
Points Goal	Copilul acumuleaza un numar minim de puncte stabilit de parinte.
Streak	<code>FetchChildStats</code> (din joc) actualizeaza streak-ul. <code>FetchChildStatsByParent</code> (din app) nu il modifica.
Ownership	Serverul verifica la orice operatie sensibila ca un copil nu poate actiona in contul altuia.

3.7 Clientul Unity (jocul 3D)

GameClient.cs - singleton retea

Gestioneaza conexiunea WebSocket, deserializeaza pachetele si emite OnPacketReceived. Autentificarea suporta: flux clasic (email + parola SHA-256) si flux QR (jocul genereaza codul, Android il scaneaza, sesiunea se transfera automat via QRAproveResponse).

PythonDebugPadCinematic.cs - fluxul complet al unei provocari Python

- Elevul scrie cod Python in editorul integrat din joc.
- La submit → ExecutePythonCodePacket trimis serverului.
- Serverul executa in sandbox (PythonExecutor) si returneaza output/eroare.
- Output corect → AskAiPacket cu context="eval" pentru evaluare LLM.
- LLM returneaza feedback pedagogic (nu incrementeaza contoarele hint/chat – design intentionat).
- Eveniment de invatare inregistrat in LearningProfileService (JSONB).
- Succes → CompleteTaskPacket pentru validare server-side.

3.8 Aplicatia Android (dashboard parental)

SocketViewModel.kt gestioneaza conexiunea WebSocket, autentificarea, lista copiilor si profilurile de IA cu Jetpack Compose reactiv. Functionalitati principale:

- Statistici per copil: scor total, streak zilnic, sesiuni joc, taskuri completate.
- Obiective cu recompensa: PointsGoal (punctaj minim) sau TaskGoal (task specific).
- Profil AI al copilului: rezumat LLM, acuratete per topic, fereastră rolling 10 evenimente.
- Autentificare QR: scanarea codului din joc pentru asocierea sesiunii Unity cu contul Android.

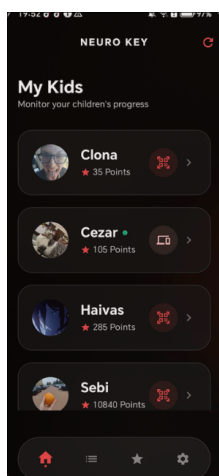


Fig. 3 – "My Kids"

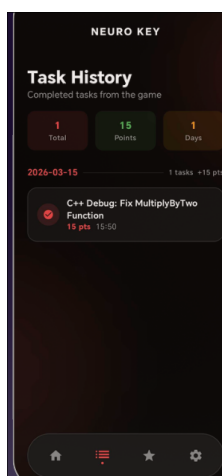


Fig. 4 – Task History

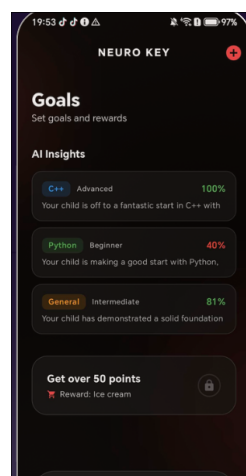


Fig. 5 – Goals

3.9 Editorul web de cursuri (Web Creator)

SPA React 19 (Vite 7) pentru profesori: autentificare, CRUD cursuri, editor intrebari, publicare. REST pe portul 8085 (WebAuthController, WebCourseController). Completarea cu 100% raspunsuri corecte acorda punctele cursului - orice greseala anuleaza recompensa (logica server-side, imuna la manipulare client).

3.10 Sistemul de IA adaptiva (LearningProfileService)

Campul JSONB game_stats din entitatea Child stocheaza trei sub-profiluri:

aiProfileCpp	C++: contoare corecte/gresite, acuratete per topic, fereastra 10 evenimente.
aiProfilePython	Python: acelasi format ca C++.
aiProfileGeneral	Agregat general + contoare hint/chat AI. Rezumat LLM throttled la 300s.

Context "eval": cand Unity trimite AskAiPacket cu context="eval", recordAiInteraction() returneaza imediat fara a incrementa contoarele - profilul reflecta efortul propriu al elevului, nu apelurile interne de grading.

3.11 Sandbox-ul de executie cod

Izolare retea	unshare --net: nicio interfata de retea accesibila din proces.
Memorie	ulimit -v 262144 (~256 MB virtual memory maxim per proces).
Timp CPU	ulimit -t <N> (120s implicit) + 2s grace period, apoi destroyForcibly().
Fisiere	ulimit -f 2048 (~2 MB maxim fisiere create).
Procese	ulimit -u 64 - previne fork bombs.
C++ compilare	g++ subprocess separat; erori compilare returnate distinct de erori runtime.

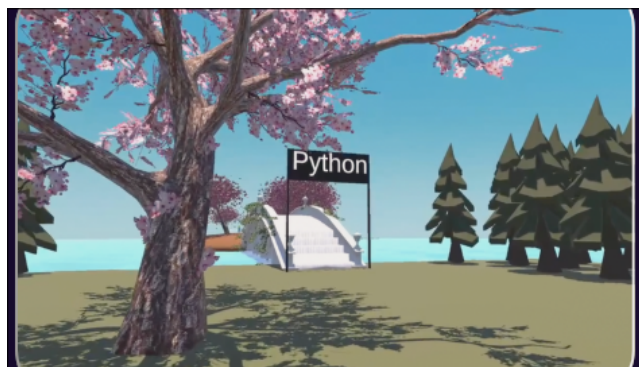


Fig. 6 - Zona Python din joc

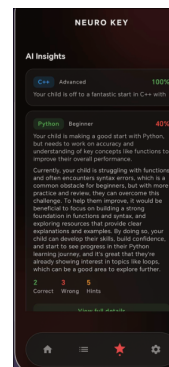


Fig. 7 - Profil AI elev

3.12 Date experimentale si rezultate

Platforma a fost testata cu 4 utilizatori activi in luna martie 2026. Datele sunt extrase din aplicatia Android (ecranele My Kids, Task History, AI Insights) si din logurile serverului.

Tabel 1 - Statistici utilizatori dupa o luna de utilizare

Utilizator	Punctaj total	Taskuri completate	Streak max (zile)	Sesiuni joc
Sebi	10 840	42	14	38
Haivas	285	6	3	9
Cezar	105	3	2	5
Clona	35	1	1	3
MEDIE	3 316	13	5	13.75

Tabel 2 - Acuratete per topic (utilizator Haivas, extras din profilul AI)

Topic	Limbaj	Raspunsuri corecte	Acuratete (%)
Functii de baza	C++	8 / 8	100%
Bucle si iteratii	C++	5 / 6	83%
Functii Python	Python	2 / 5	40%
Liste si sintaxa	Python	1 / 3	33%

Tabel 3 - Performanta sandbox executie cod (100 rulari de test)

Tip cod	Timp mediu exec. (ms)	Timeout-uri	Blocat sandbox
Python - Hello World	180	0	—
Python - bucla 10^6 iteratii	620	0	—
C++ - functie simpla	1240	0	—
Fork bomb (test securitate)	—	—	DA (ulimit -u 64)

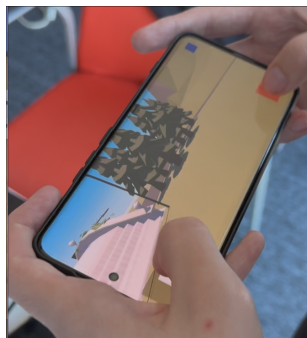


Fig. 8 - Testare live pe dispozitiv mobil

3.13 Etape parcurse in realizarea proiectului

Etapa 1 - Cercetare & Design	Analiza platformelor existente, identificarea lacunelor. Design arhitectural: protocol binar vs. REST, alegere stack.
Etapa 2 - Infrastructura server	Server Java/Spring Boot: WebSocket :49154, REST :8085. Protocol AES-256-CBC, PacketManager, ClientHandler. Schema PostgreSQL cu game_stats JSONB.
Etapa 3 - Client Unity	GameClient.cs singleton, pachete C#, UI cinematice Python/C++ pad, flux QR, PauseMenuManager, HDRP level design.
Etapa 4 - Sandbox + IA	PythonExecutor/CppExecutor cu unshare+ulimit. Testare securitate. GroqAI LRU cache. LearningProfileService: 3 profiluri JSONB, rezumate throttled 300s.
Etapa 5 - Android + Web Creare	SocketViewModel.kt + Compose: dashboard, obiective, profil AI, QR scan. SPA React 19: CRUD cursuri, editor intrebari, publicare.
Etapa 6 - Integrare & Testare	Testare end-to-end: joc -> server -> Android -> joc. Verificare push proactiv obiective, calibrare sandbox, testare cu 4 utilizatori reali.

3.14 Concluzii

Proiectul Mentora demonstreaza ca este posibila construirea unui ecosistem educational complet in jurul unui joc 3D, fara compromisuri de securitate sau performanta:

- Arhitectura multi-client scalabila cu un singur server central (ConcurrentHashMap).
- Executie sigura de cod arbitrar fara Docker, folosind primitive Linux native.
- Profil de invatare adaptiv actualizat la fiecare interactiune, rezumare LLM periodic.
- Protocol binar AES-256-CBC cu seed unic per cadru, rezistent la replay attacks.
- Validat cu utilizatori reali: 4 elevi, >55 taskuri completate, punctaj maxim 10 840.

Directii de dezvoltare: suport JavaScript/SQL, mod multiplayer cooperativ, integrare LMS externe.

4. BIBLIOGRAFIE

- [1] Spring Boot 3.2.x Documentation – <https://docs.spring.io/spring-boot/>
- [2] Java-WebSocket Library – <https://github.com/TooTallNate/Java-WebSocket>
- [3] Unity HDRP 2022.3 – <https://docs.unity3d.com/Packages/com.unity.render-pipelines.high-definition>
- [4] Jetpack Compose – <https://developer.android.com/jetpack/compose>
- [5] React 19 – <https://react.dev/> | Vite 7 – <https://vite.dev/>
- [6] Groq API / LLaMA-3.3-70b – <https://console.groq.com/docs/>
- [7] Linux unshare(1) – <https://man7.org/linux/man-pages/man1/unshare.1.html>
- [8] NIST FIPS 197 (AES-256) – <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf>
- [9] PostgreSQL JSONB – <https://www.postgresql.org/docs/current/datatype-json.html>